



SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

WEEK 3 FOCUS

Form Validation & Submission Interception

Muneeb Ur Rehman

MERN STACK INTERN

1. Objective

The purpose of this exercise was to build a client-side user registration form that intercepts standard browser submission behavior. The goal was to programmatically read the form data and enforce custom security rules—specifically password length and password matching—before allowing the form to process.

2. Theoretical Background

Modern web applications rely heavily on client-side validation to provide immediate feedback to users and reduce unnecessary server requests. This lab utilized three core JavaScript concepts:

- **Preventing Default Behavior:** Using `event.preventDefault()` stops the browser from reloading the page upon form submission, allowing JavaScript to handle the data flow.
- **The FormData Object:** A built-in browser API (`new FormData()`) that automatically extracts key-value pairs from a form, eliminating the need to query the DOM for every individual input field.
- **Custom Validation:** Writing conditional logic to enforce rules that native HTML5 validation cannot handle on its own (such as verifying that two separate password fields contain identical strings).

3. Methodology

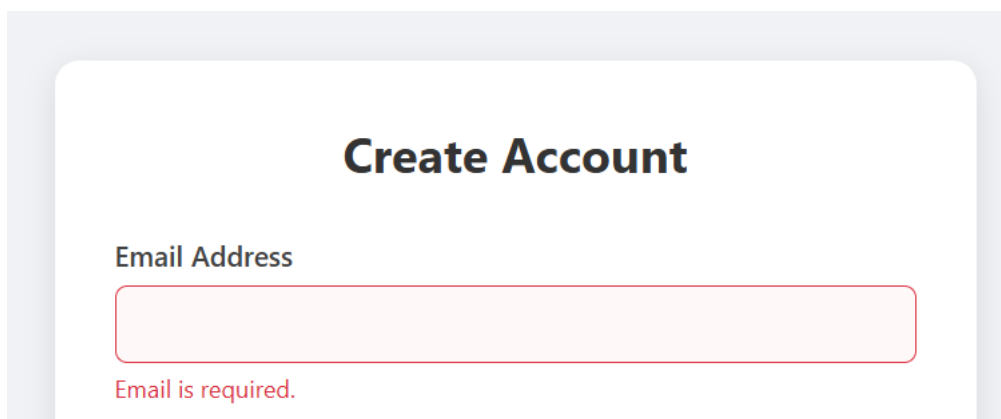
A. User Interface Construction A registration form was created using HTML and styled with a modern, centered CSS layout. The `<form>` tag included the `novalidate` attribute to disable the browser's default popup bubbles, allowing for a fully customized inline error system. Empty `<div>` elements were placed under each input to serve as containers for potential error messages.

B. Event Interception & Data Retrieval An event listener was attached to the form to listen for the `submit` event. Upon submission:

1. `event.preventDefault()` was immediately triggered.
2. Previous error messages and red borders were cleared using a helper function.
3. A `FormData` object was instantiated to capture the user's input values (email, password, and `confirmPassword`).

C. Validation Logic Routing A boolean flag (`isFormValid`) was set to true by default. The script then ran the data through a series of conditional checks:

- Checked if the email field was empty.



Create Account

Email Address

Email is required.

- Checked if the password string length was strictly less than 8 characters.

Create Account

Email Address

Password

Password must be at least 8 characters long.

Confirm Password

Register

- Checked if the `password` value strictly matched the `confirmPassword` value.

Password

Password must be at least 8 characters long.

Confirm Password

Passwords do not match.

Register

If any check failed, `isFormValid` was flipped to false, and a specific error message was injected into the DOM beneath the corresponding input field.

4. Results and Observations

When testing the application in the browser, the validation logic performed exactly as intended:

- **Submission Prevention:** Pressing the "Register" button never caused the page to reload, confirming the default event was successfully intercepted.
- **Length Validation:** Entering a password of 5 characters immediately highlighted the password field in red and displayed the message "Password must be at least 8 characters long."
- **Matching Validation:** Entering valid but mismatched passwords triggered an error on the confirmation field stating "Passwords do not match," blocking submission.
- **Successful Execution:** When all criteria were met (valid email, 8+ character password, and matching confirmation), all errors cleared, a green "Registration successful!" message appeared, and the form fields were wiped clean using `form.reset()`.

5. Conclusion

The lab successfully demonstrated how to take complete control of HTML forms using JavaScript. By leveraging `e.preventDefault()` and the `FormData` API, we created a seamless, user-friendly registration process. The implementation of inline error messages provides a vastly superior User Experience (UX) compared to native browser alerts, confirming the importance of custom client-side validation in modern web development.

